

Numerical Methods for the Solution of PDEs

Laboratory with deal.II — www.dealii.org

Numerical Solution of PDEs using the Finite Element Method

Luca Heltai <luca.heltai@unipi.it>

A posteriori error estimates and adaptive meshes

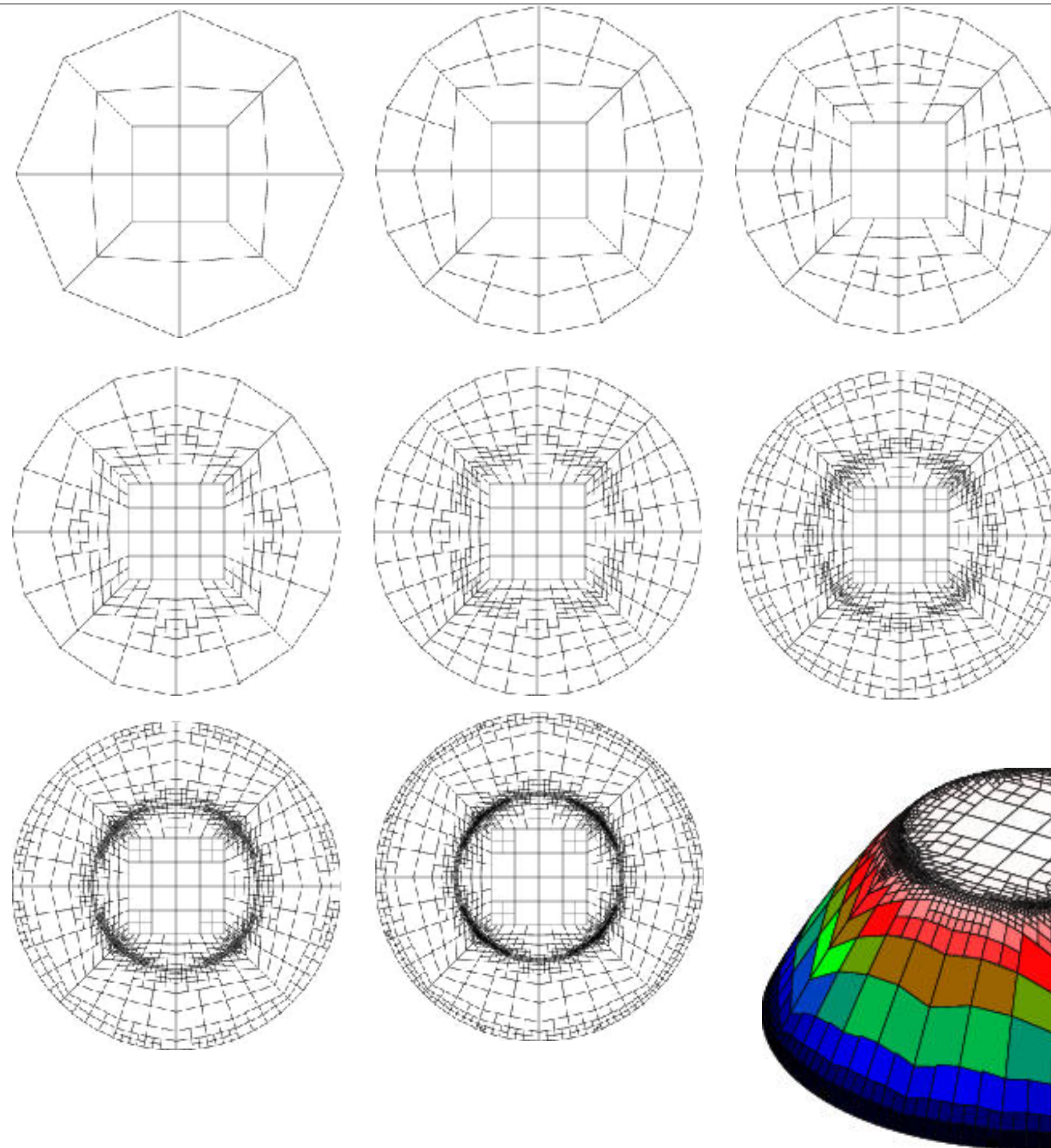
<https://luca-heltai.github.io/nmpde>
<https://github.com/luca-heltai/nmpde>



Adaptive mesh refinement

- Typical steps to perform adaptivity
 - Solve (non-)linear system
 - Estimate error
 - Mark cells
 - Refine/coarsen
 - Interpolate original solution to new mesh

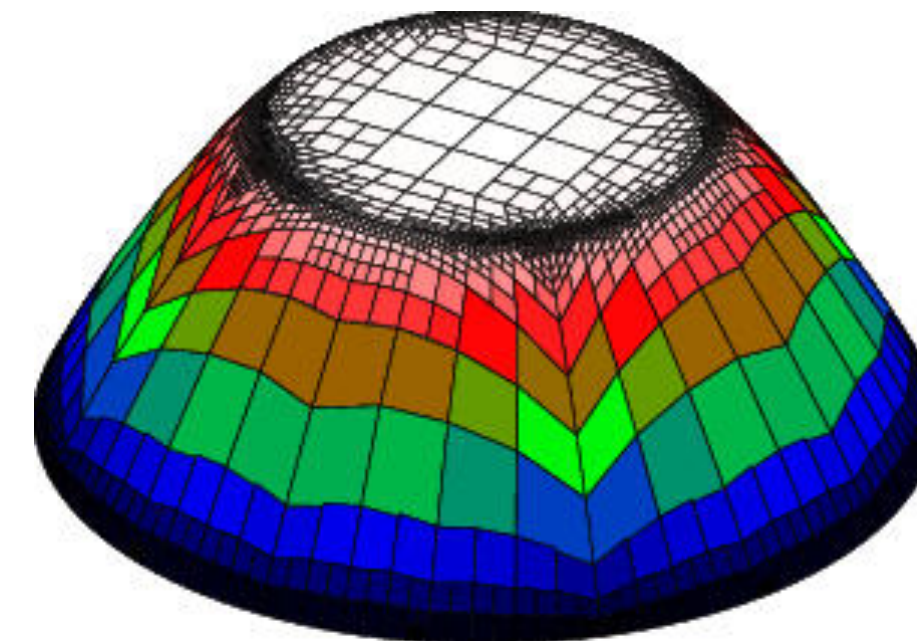
Need an error indicator on each cell without knowing the exact solution. η_K



$$\nabla \cdot a(\mathbf{x}) \nabla u(\mathbf{x}) = 1 \quad \text{in } \Omega$$

$$u = 0 \quad \text{on } \partial\Omega$$

$$a(\mathbf{x}) = \begin{cases} 20 & \text{if } |\mathbf{x}| < 0.5 \\ 1 & \text{otherwise} \end{cases}$$



Adaptive mesh refinement

Error estimate for QI/PI elements applied to Laplace problem:

$$\|u - u^h\|_{H^1} \equiv \|e\|_{H^1} \leq C h_{max} \|u\|_{H^2}$$

this error depends on the largest element size and the global norm of the solution.

To reduce error (increase accuracy) one can refine the mesh size.

more precisely...

$$\|e\|_{H^1}^2 \leq C^2 \sum_K h_K^2 |u|_{H^2(K)}^2$$

Thus one needs to **make mesh finer where the local H^2 semi-norm is large**.

But apart from some special cases we don't know the exact solution u !

Thus we need to create meshes iteratively (adaptively).

Optimal strategy is to equilibrate the error $e_K := C h_K |u|_{H^2(K)}$

$$\begin{aligned} \|u\|_{H^2(K)}^2 &:= \int_K u^2 + |\nabla u|^2 + |\nabla^2 u|^2 \\ |u|_{H^2(K)}^2 &:= \int_K |\nabla^2 u|^2 \end{aligned}$$

That is, we want to choose $h_K \sim \frac{1}{|u|_{H^2(K)}}$



a-posteriori error estimation

$$||e||_{H^1(\Omega)}^2 \leq C \sum_K e_K^2$$

cell-wise error indicators

$$e_K = h_K ||\nabla^2 u||_K$$

(wrong) idea:

$$e_K \approx h_K ||\nabla^2 u^h||_K$$

will not work as linear elements have zero second derivatives within the element and first derivatives have jumps on the interfaces

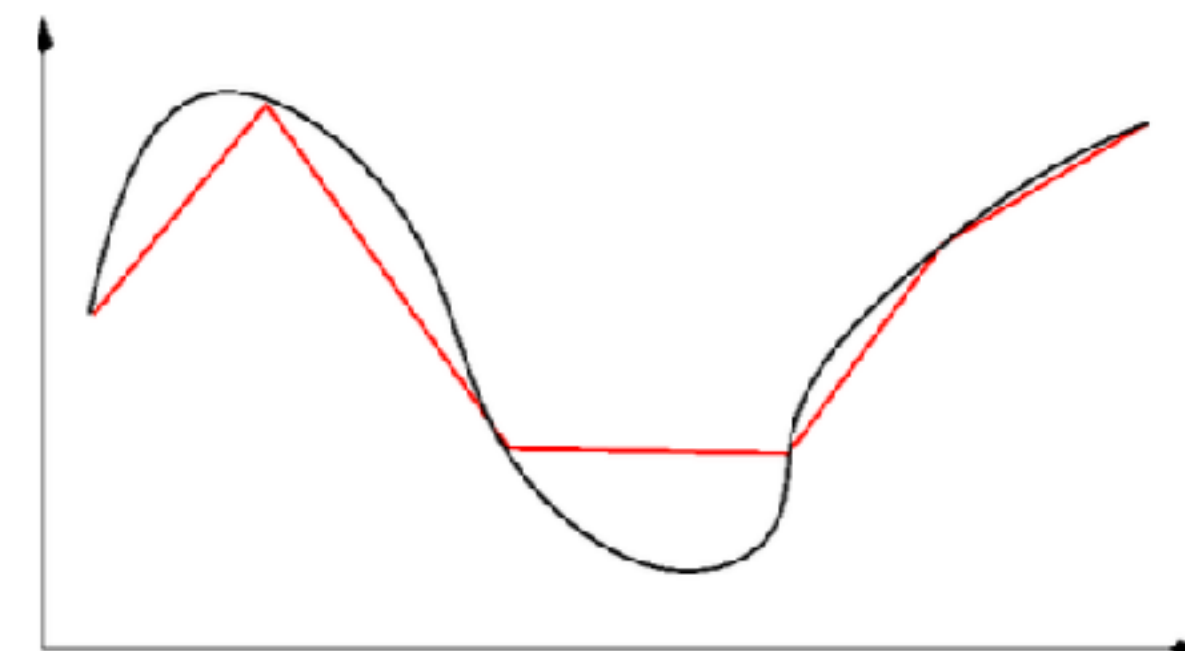
a better idea (in 1D) to approximate second derivatives at interface i :

$$\nabla^2 u \approx \frac{\nabla u^h(x^+) - \nabla u^h(x^-)}{h} =: \frac{[[\nabla u^h]]_i}{h}$$

can generalize to:

$$||\nabla^2 u||_K^2 \approx \sum_{i \in \partial K} \frac{[[\nabla u^h]]_i^2}{h}$$

use jump in gradient as an indicator of the second derivative at vertices



a-posteriori error estimation

As a result, the simplest and most widely used Kelly error **indicator** in 2D/3D follows:

$$e_K^2 = h_K^2 \|\nabla^2 u\|_K^2 \approx h_K \int_{\partial K} |[\![\nabla u \cdot \mathbf{n}]\!]|^2 ds =: \eta_K^2$$

For the Laplace equation, **Kelly, de Gago, Zienkiewicz, Babushka (1983)** proved that

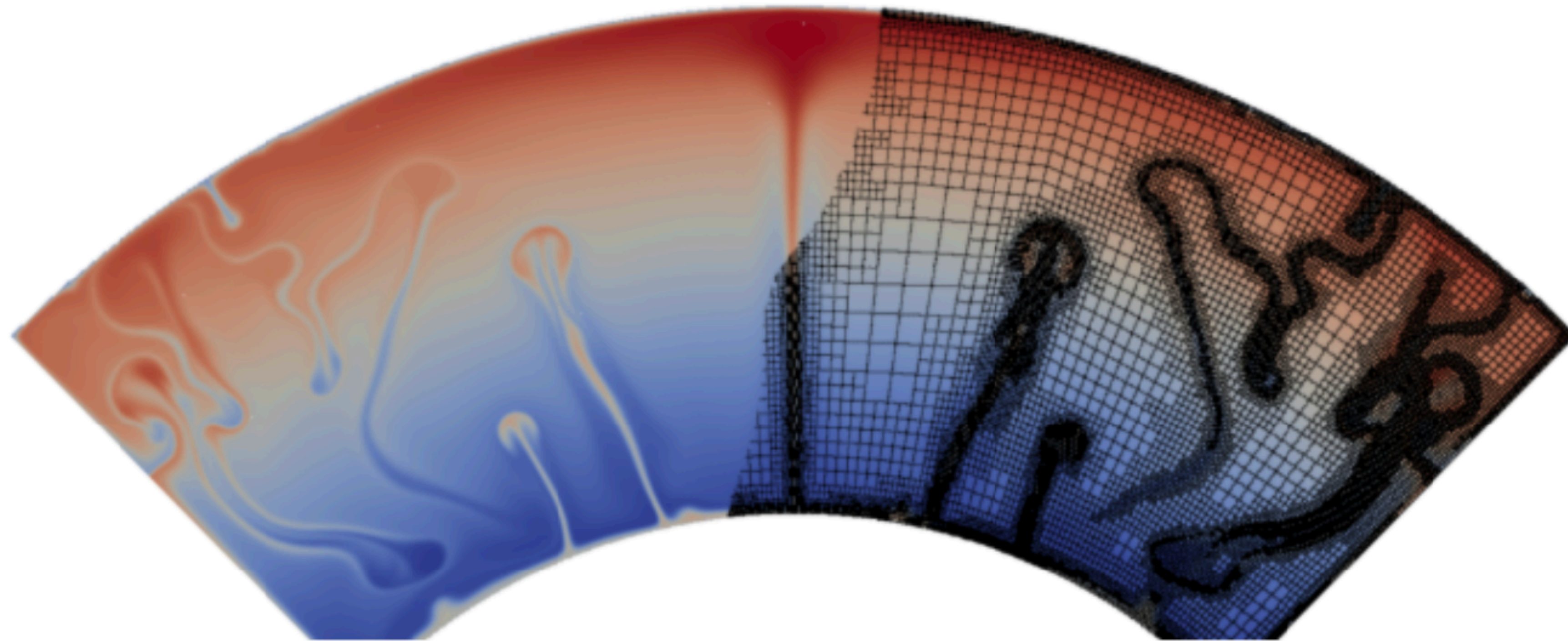
$$\|\nabla [u - u^h]\|^2 \leq C \sum_K \eta_K^2 \quad \text{a-posteriori error estimator} \quad (*)$$

Note I:

“**estimator**” is always a proven upper bound of error (*), whereas “**indicator**” is our best guess of error per cell which may not be an upper bound in the sense (*), but may still work well for considered equations and/or FE space.



Adaptive mesh refinement



Refine where things are happening!
UNIVERSITÀ DI PISA



Basic AFEM algorithm

- SOLVE-ESTIMATE-MARK-REFINE
 - On the current mesh, solve the problem
 - Estimate the error per cell (Exact, Kelly, Residual, etc.)
 - Mark cells according to given criterion (estimator is greater than a tolerance, or fraction of cells with largest error, or ...)
 - Refine the marked cells
- Repeat until tolerance met, or max number of cycles



deal.II classes

- Error estimate is problem dependent:
 - Approximate gradient jumps: `KellyErrorEstimator` class
 - Approximate local norm of gradient: `DerivativeApproximation` class
 - ... or something else
- Cell marking strategy:
 - `GridRefinement::refine_and_coarsen_fixed_number(...)`
 - `GridRefinement::refine_and_coarsen_fixed_fraction(...)`
 - `GridRefinement::refine_and_coarsen_optimize(...)`
- Refine/coarsen grid: `triangulation.execute_coarsening_and_refinement ()`
- Transferring the solution: `SolutionTransfer` class (discussed later)

